UNIVERSIDAD NACIONAL DE COLOMBIA

Programación Orientada a Objetos 2026-2S

Actividad 3

Estudiante: Juan Miguel Fernandez Gonzalez
Correo: jufernandezgo@unal.edu.co
Docente: Walter Hugo Arboleda Mazo

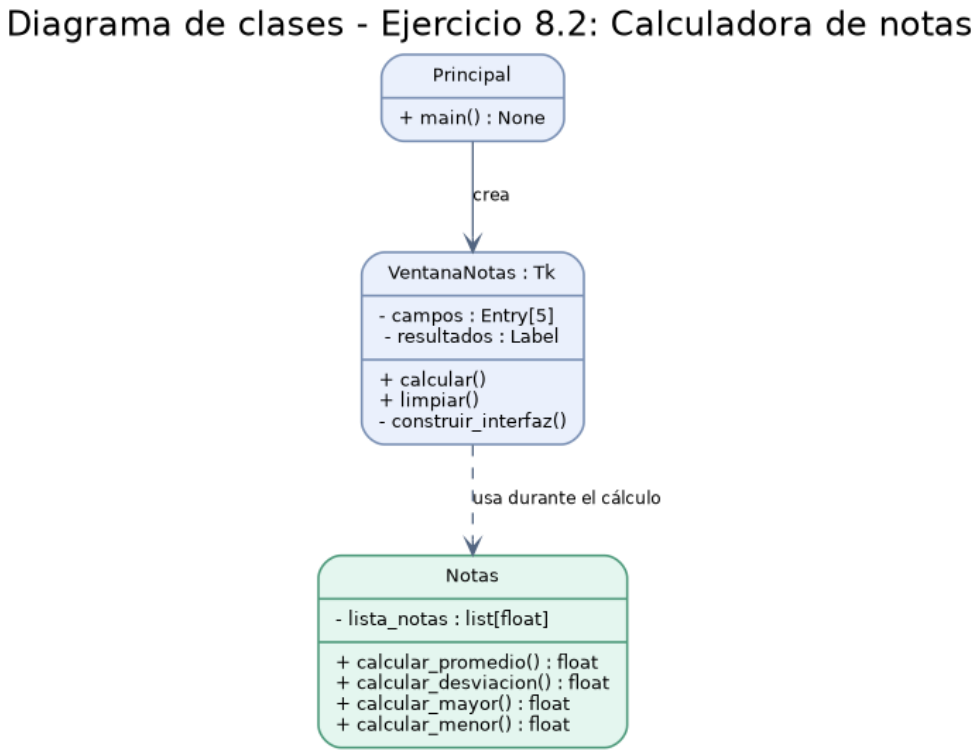
Octubre de 2026

Ejercicio Resuelto 8.2

Se requiere desarrollar un programa con interfaz gráfica que permita ingresar cinco notas obtenidas por un estudiante. En la parte inferior de la ventana se muestran el promedio de las notas, la desviación estándar, la nota mayor y la nota menor. La ampliación propuesta solicita generar alertas para entradas no numéricas y hacer obligatorio el ingreso de las cinco notas.

Código desarrollado en Python con Tkinter.

Diagrama de clases

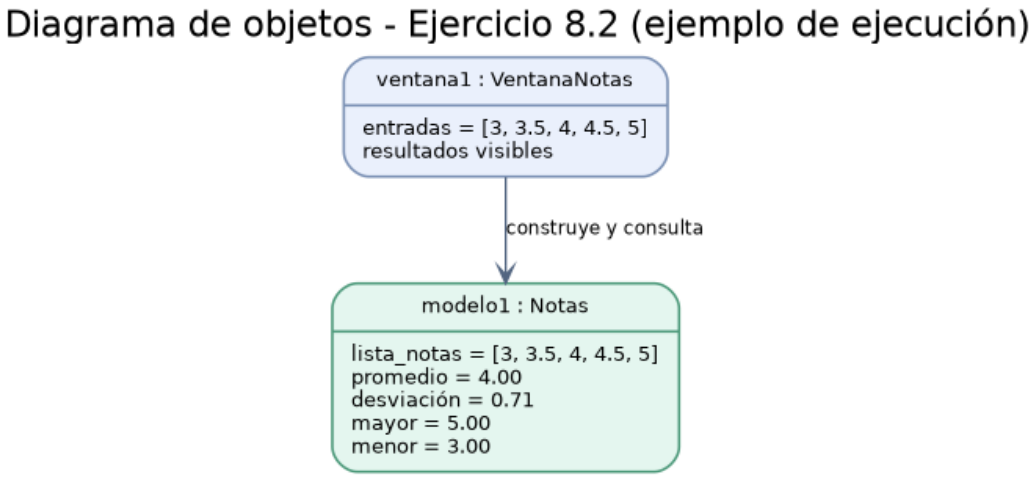


Ejercicio Resuelto 8.2 - Interfaz de usuario

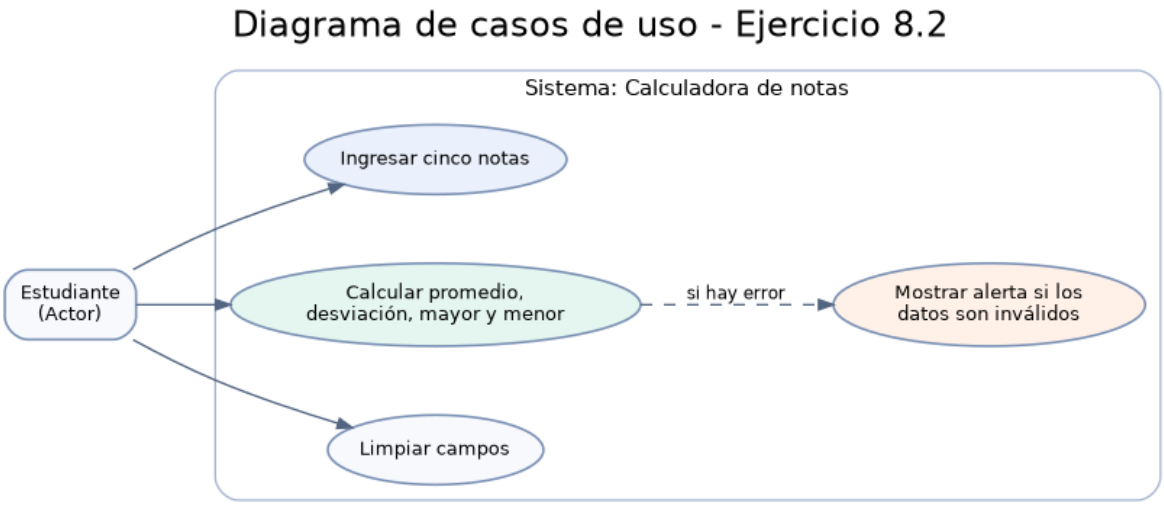


Ejercicio Resuelto 8.2 - Diagramas complementarios

Diagrama de objetos



Casos de uso



Código fuente (Python) - Ejercicio 8.2: notas.py

```
"""Ejercicio 8.2 adaptado a Python/Tkinter: calculadora de cinco notas.

Basado en el enunciado del libro Ejercicios de programación orientada a objetos
con Java y UML, ejercicio 8.2 (pp. 483-494). La desviación estándar calculada
es poblacional, siguiendo la división entre el número total de notas.
"""
from __future__ import annotations

import math
import tkinter as tk
from tkinter import messagebox, ttk
from typing import Sequence


class Notas:
    """Modelo que almacena cinco notas y calcula indicadores estadísticos."""

    def __init__(self, lista_notas: Sequence[float] | None = None) -> None:
        self.lista_notas = list(lista_notas) if lista_notas is not None else []
        if self.lista_notas and len(self.lista_notas) != 5:
            raise ValueError("Se requieren exactamente cinco notas.")

    def _verificar_cinco(self) -> None:
        if len(self.lista_notas) != 5:
            raise ValueError("Se requieren exactamente cinco notas.")

    def calcular_promedio(self) -> float:
        self._verificar_cinco()
        return sum(self.lista_notas) / len(self.lista_notas)

    def calcular_desviacion(self) -> float:
        self._verificar_cinco()
        promedio = self.calcular_promedio()
        varianza = sum((nota - promedio) ** 2 for nota in self.lista_notas) / len(self.lista_notas)
        return math.sqrt(varianza)

    def calcular_mayor(self) -> float:
        self._verificar_cinco()
        return max(self.lista_notas)

    def calcular_menor(self) -> float:
        self._verificar_cinco()
        return min(self.lista_notas)


class VentanaNotas(tk.Tk):
    """Interfaz gráfica para registrar las notas y presentar los resultados."""

    BG = "#F3F6FB"
    NAVY = "#172B4D"
    BLUE = "#2F65D5"
    MUTED = "#63738A"
    WHITE = "#FFFFFF"

    def __init__(self) -> None:
        super().__init__()
        self.title("Actividad 3 | Calculadora de notas")
        self.geometry("900x680")
        self.minsize(820, 640)
        self.configure(bg=self.BG)
        self._configurar_estilos()
        self.campos: list[ttk.Entry] = []
        self.resultados: dict[str, ttk.Label] = {}
```

Código fuente (Python) - Ejercicio 8.2: notas.py - continuación (2/4)

```
        self._construir_interfaz()

    def _configurar_estilos(self) -> None:
        style = ttk.Style(self)
        style.theme_use("clam")
        style.configure("TFrame", background=self.BG)
        style.configure("Card.TFrame", background=self.WHITE)
        style.configure("TLabel", background=self.BG, foreground=self.NAVY, font=("Segoe UI", 10))
        style.configure(
            "Title.TLabel", background=self.NAVY, foreground=self.WHITE,
            font=("Segoe UI", 22, "bold")
        )
        style.configure(
            "Subtitle.TLabel", background=self.NAVY, foreground="#D8E4F7",
            font=("Segoe UI", 10)
        )
        style.configure(
            "Section.TLabel", background=self.WHITE, foreground=self.NAVY,
            font=("Segoe UI", 12, "bold")
        )
        style.configure("Field.TLabel", background=self.WHITE, foreground=self.MUTED, font=("Segoe UI", 10))
        style.configure(
            "Value.TLabel", background=self.WHITE, foreground=self.BLUE,
            font=("Segoe UI", 19, "bold")
        )
        style.configure("TEntry", padding=9, fieldbackground="#FBFCFE", font=("Segoe UI", 11))
        style.configure(
            "Primary.TButton", background=self.BLUE, foreground=self.WHITE,
            padding=(15, 10), font=("Segoe UI", 10, "bold"), borderwidth=0
        )
        style.map("Primary.TButton", background=[("active", "#214EAD")])
        style.configure(
            "Secondary.TButton", background="#E9EEF7", foreground=self.NAVY,
            padding=(15, 10), font=("Segoe UI", 10, "bold"), borderwidth=0
        )

    def _construir_interfaz(self) -> None:
        header = tk.Frame(self, bg=self.NAVY, height=110)
        header.pack(fill="x")
        header.pack_propagate(False)
        tk.Label(header, text="Calculadora de notas", bg=self.NAVY, fg=self.WHITE,
                 font=("Segoe UI", 23, "bold")).pack(anchor="w", padx=30, pady=(21, 2))
        tk.Label(header, text="Registra cinco notas y consulta el resumen estadístico.",
                 bg=self.NAVY, fg="#D8E4F7", font=("Segoe UI", 10)).pack(anchor="w", padx=32)

        body = ttk.Frame(self, padding=(24, 22, 24, 18))
        body.pack(fill="both", expand=True)
        body.columnconfigure(0, weight=1, uniform="col")
        body.columnconfigure(1, weight=1, uniform="col")
        body.rowconfigure(0, weight=1)

        left = tk.Frame(body, bg=self.WHITE, highlightbackground="#E0E7F0", highlightthickness=1)
        left.grid(row=0, column=0, sticky="nsew", padx=(0, 10))
        right = tk.Frame(body, bg=self.WHITE, highlightbackground="#E0E7F0", highlightthickness=1)
        right.grid(row=0, column=1, sticky="nsew", padx=(10, 0))

        tk.Label(left, text="ENTRADA DE DATOS", bg=self.WHITE, fg=self.NAVY,
                 font=("Segoe UI", 12, "bold")).pack(anchor="w", padx=22, pady=(24, 4))
        tk.Label(left, text="Completa todos los campos con valores numéricos.",
                 bg=self.WHITE, fg=self.MUTED, font=("Segoe UI", 9), wraplength=320,
                 justify="left").pack(anchor="w", padx=22, pady=(0, 18))

        form = tk.Frame(left, bg=self.WHITE)
```

Código fuente (Python) - Ejercicio 8.2: notas.py - continuación (3/4)

```
        form.pack(fill="x", padx=22)
        for i in range(5):
            row = tk.Frame(form, bg=self.WHITE)
            row.pack(fill="x", pady=6)
            tk.Label(row, text=f"Nota {i + 1}", bg=self.WHITE, fg=self.NAVY,
                     font=("Segoe UI", 10, "bold"), width=10, anchor="w").pack(side="left")
            entry = ttk.Entry(row, width=18)
            entry.pack(side="right", fill="x", expand=True, ipady=2)
            self.campos.append(entry)

        buttons = tk.Frame(left, bg=self.WHITE)
        buttons.pack(fill="x", padx=22, pady=(20, 12))
        ttk.Button(buttons, text="Calcular resultados", style="Primary.TButton",
                   command=self.calcular).pack(fill="x", pady=(0, 9))
        ttk.Button(buttons, text="Limpiar campos", style="Secondary.TButton",
                   command=self.limpiar).pack(fill="x")
        tk.Label(left, text="Los campos vacíos o no numéricos generan una alerta.",
                 bg=self.WHITE, fg=self.MUTED, font=("Segoe UI", 8), wraplength=310,
                 justify="left").pack(anchor="w", padx=22, pady=(13, 20))

        tk.Label(right, text="RESUMEN ESTADÍSTICO", bg=self.WHITE, fg=self.NAVY,
                 font=("Segoe UI", 12, "bold")).pack(anchor="w", padx=22, pady=(24, 5))
        tk.Label(right, text="Promedio, desviación estándar, nota mayor y menor.",
                 bg=self.WHITE, fg=self.MUTED, font=("Segoe UI", 9), wraplength=320,
                 justify="left").pack(anchor="w", padx=22, pady=(0, 15))

        for title, key in [("Promedio", "promedio"), ("Desviación estándar", "desviacion"),
                           ("Nota mayor", "mayor"), ("Nota menor", "menor")]:
            card = tk.Frame(right, bg="#F7F9FD", highlightbackground="#E7EDF6", highlightthickness=1)
            card.pack(fill="x", padx=22, pady=6)
            tk.Label(card, text=title, bg="#F7F9FD", fg=self.MUTED,
                     font=("Segoe UI", 9)).pack(anchor="w", padx=13, pady=(9, 0))
            value = tk.Label(card, text="--", bg="#F7F9FD", fg=self.BLUE,
                             font=("Segoe UI", 18, "bold"))
            value.pack(anchor="w", padx=13, pady=(0, 8))
            self.resultados[key] = value

        self.estado = tk.Label(self, text="Listo para ingresar las notas", bg=self.BG,
                               fg=self.MUTED, font=("Segoe UI", 9), anchor="w")
        self.estado.pack(fill="x", padx=28, pady=(0, 12))

    @staticmethod
    def _convertir_numero(texto: str) -> float:
        return float(texto.strip().replace(",", "."))

    def calcular(self) -> None:
        valores: list[float] = []
        try:
            for indice, campo in enumerate(self.campos, start=1):
                texto = campo.get().strip()
                if not texto:
                    raise ValueError(f"Debes ingresar la Nota {indice}.")
                try:
                    valor = self._convertir_numero(texto)
                except ValueError as exc:
                    raise ValueError(f"La Nota {indice} debe ser un número válido.") from exc
                if not math.isfinite(valor):
                    raise ValueError(f"La Nota {indice} debe ser un número finito.")
                valores.append(valor)
            modelo = Notas(valores)
        except ValueError as exc:
            messagebox.showerror("Datos inválidos", str(exc), parent=self)
            self.estado.configure(text="Revisa los datos marcados por la alerta", fg="#B42318")
```

Código fuente (Python) - Ejercicio 8.2: notas.py - continuación (4/4)

```
            return

        resultados = {
            "promedio": modelo.calcular_promedio(),
            "desviacion": modelo.calcular_desviacion(),
            "mayor": modelo.calcular_mayor(),
            "menor": modelo.calcular_menor(),
        }
        for clave, valor in resultados.items():
            self.resultados[clave].configure(text=f"{valor:.2f}")
        self.estado.configure(text="Cálculo completado correctamente", fg="#16803C")

    def limpiar(self) -> None:
        for campo in self.campos:
            campo.delete(0, tk.END)
        for etiqueta in self.resultados.values():
            etiqueta.configure(text="--")
        self.estado.configure(text="Campos limpios. Ingresa cinco notas para comenzar.", fg=self.MUTED)
        self.campos[0].focus_set()

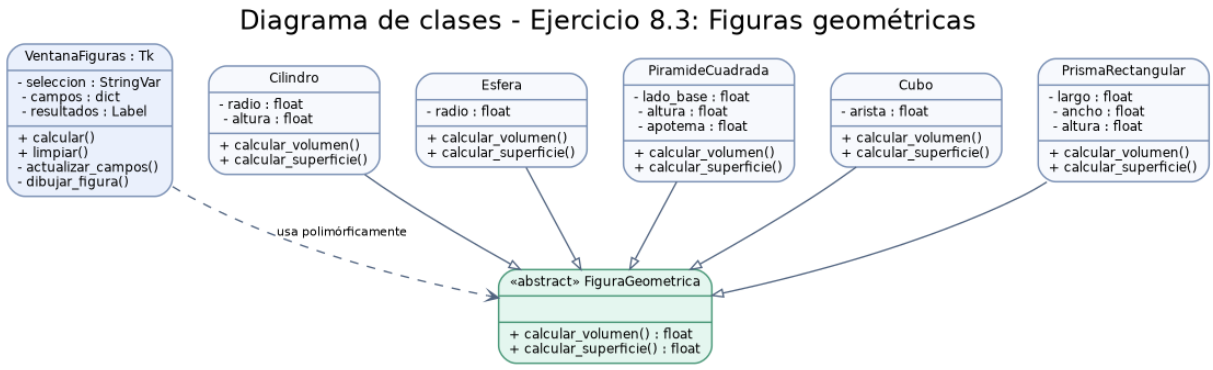

def main() -> None:
    app = VentanaNotas()
    app.mainloop()


if __name__ == "__main__":
    main()
```

Ejercicio Resuelto 8.3

Se requiere desarrollar una interfaz gráfica para calcular el volumen y la superficie de un cilindro, una esfera y una pirámide cuadrada. Como ampliación, se agregan el cubo y el prisma rectangular.

Diagrama de clases



Fórmulas implementadas

Figura	Volumen	Superficie
Cilindro	$\pi\ r^2\ h$	$2\pi r(r + h)$
Esfera	$(4/3)\pi r^3$	$4\pi r^2$
Pirámide cuadrada	$a^2h\ /\ 3$	$a^2 + 2al$
Cubo	a^3	$6a^2$
Prisma rectangular	largo × ancho × altura	$2(la + lh + ah)$

Ejercicio Resuelto 8.3 - Interfaz de usuario

Calculadora de figuras

Selecciona una figura, registra sus dimensiones y calcula sus propiedades.

DATOS DE LA FIGURA

Figura geométrica

Cilindro

Radio (cm) 3

Altura (cm) 5

Volumen: 141.37 cm³

Superficie: 150.80 cm²

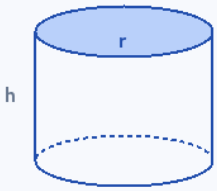
Calcular | Limpiar

Las dimensiones deben ser positivas y numéricas.

VISTA DE LA FIGURA

Esquema ilustrativo de la figura seleccionada

Cilindro

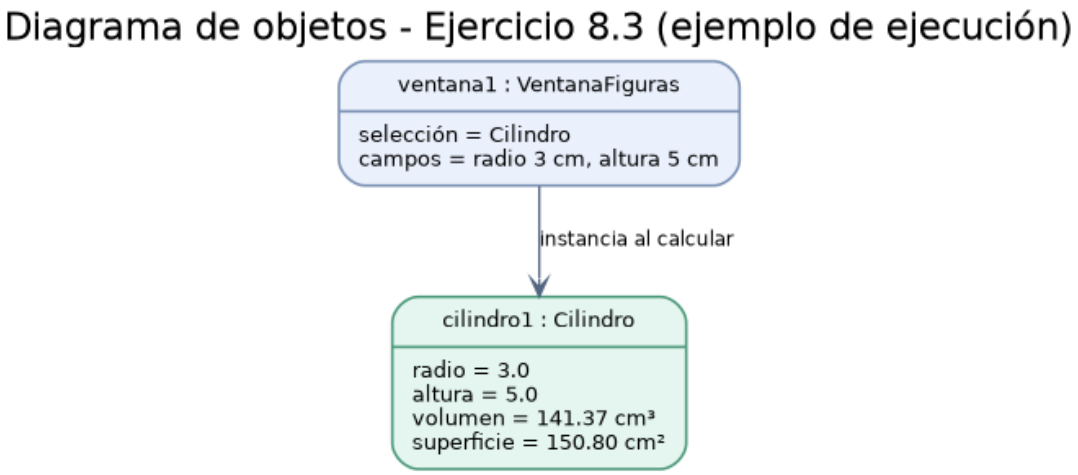


$V = \pi r^2 h$ $S = 2\pi r(r + h)$

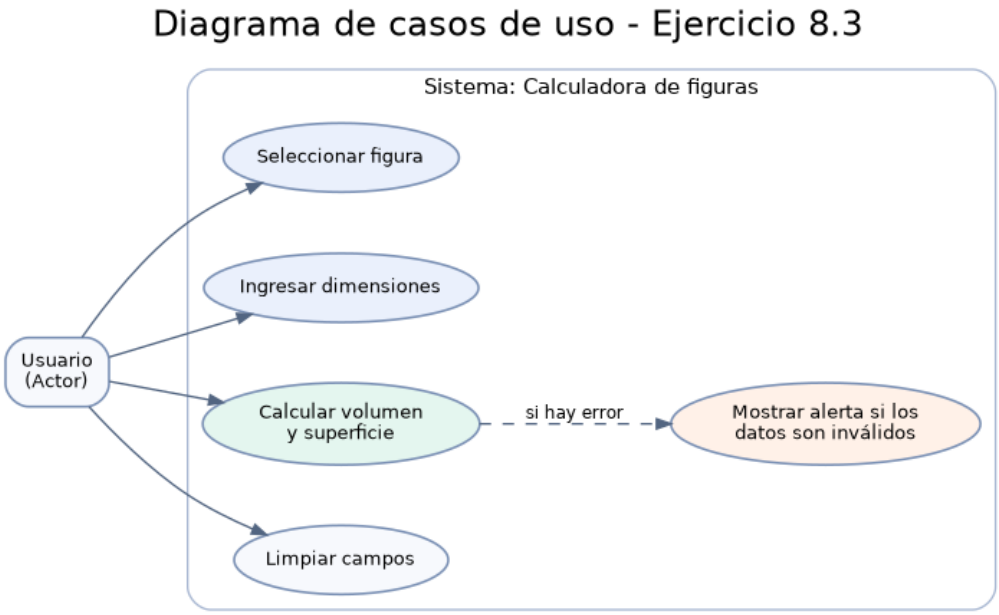
Cálculo completado: Cilindro.

Ejercicio Resuelto 8.3 - Diagramas complementarios

Diagrama de objetos



Casos de uso



Código fuente (Python) - Ejercicio 8.3: figuras.py

```
"""Ejercicio 8.3 adaptado a Python/Tkinter: cálculo de figuras geométricas.

Implementa cilindro, esfera y pirámide cuadrada, y amplía la solución con
cubo y prisma rectangular.
"""
from __future__ import annotations

import math
import tkinter as tk
from abc import ABC, abstractmethod
from tkinter import messagebox, ttk
from typing import Callable


class FiguraGeometrica(ABC):
    """Clase base abstracta para figuras con volumen y superficie."""

    @abstractmethod
    def calcular_volumen(self) -> float:
        """Calcula el volumen de la figura en cm³."""

    @abstractmethod
    def calcular_superficie(self) -> float:
        """Calcula la superficie de la figura en cm²."""


class Cilindro(FiguraGeometrica):
    def __init__(self, radio: float, altura: float) -> None:
        self.radio, self.altura = _validar_positivos(radio, altura)

    def calcular_volumen(self) -> float:
        return math.pi * self.radio**2 * self.altura

    def calcular_superficie(self) -> float:
        return 2 * math.pi * self.radio * (self.radio + self.altura)


class Esfera(FiguraGeometrica):
    def __init__(self, radio: float) -> None:
        (self.radio,) = _validar_positivos(radio)

    def calcular_volumen(self) -> float:
        return (4.0 / 3.0) * math.pi * self.radio**3

    def calcular_superficie(self) -> float:
        return 4 * math.pi * self.radio**2


class PiramideCuadrada(FiguraGeometrica):
    """Pirámide de base cuadrada; apotema corresponde a la cara lateral."""

    def __init__(self, lado_base: float, altura: float, apotema: float) -> None:
        self.lado_base, self.altura, self.apotema = _validar_positivos(lado_base, altura, apotema)

    def calcular_volumen(self) -> float:
        return (self.lado_base**2 * self.altura) / 3.0

    def calcular_superficie(self) -> float:
        return self.lado_base**2 + 2 * self.lado_base * self.apotema


class Cubo(FiguraGeometrica):
    def __init__(self, arista: float) -> None:
```

Código fuente (Python) - Ejercicio 8.3: figuras.py - continuación (2/6)

```
        (self.arista,) = _validar_positivos(arista)

    def calcular_volumen(self) -> float:
        return self.arista**3

    def calcular_superficie(self) -> float:
        return 6 * self.arista**2


class PrismaRectangular(FiguraGeometrica):
    def __init__(self, largo: float, ancho: float, altura: float) -> None:
        self.largo, self.ancho, self.altura = _validar_positivos(largo, ancho, altura)

    def calcular_volumen(self) -> float:
        return self.largo * self.ancho * self.altura

    def calcular_superficie(self) -> float:
        return 2 * (self.largo * self.ancho + self.largo * self.altura + self.ancho * self.altura)


def _validar_positivos(*valores: float) -> tuple[float, ...]:
    if any(not math.isfinite(v) or v <= 0 for v in valores):
        raise ValueError("Todas las dimensiones deben ser números mayores que cero.")
    return tuple(valores)


class VentanaFiguras(tk.Tk):
    """GUI que crea las figuras según la opción seleccionada y gestiona eventos."""

    BG = "#F3F6FB"
    NAVY = "#172B4D"
    BLUE = "#2F65D5"
    MUTED = "#63738A"
    WHITE = "#FFFFFF"

    DEFINICIONES: dict[str, tuple[tuple[str, str], ...]] = {
        "Cilindro": (("radio", "Radio (cm)"), ("altura", "Altura (cm)")),
        "Esfera": (("radio", "Radio (cm)"),),
        "Pirámide cuadrada": (
            ("lado_base", "Lado de la base (cm)"),
            ("altura", "Altura (cm)"),
            ("apotema", "Apotema lateral (cm)"),
        ),
        "Cubo": (("arista", "Arista (cm)"),),
        "Prisma rectangular": (("largo", "Largo (cm)"), ("ancho", "Ancho (cm)"), ("altura", "Altura (cm)")),
    }

    FABRICAS: dict[str, Callable[..., FiguraGeometrica]] = {
        "Cilindro": Cilindro,
        "Esfera": Esfera,
        "Pirámide cuadrada": PiramideCuadrada,
        "Cubo": Cubo,
        "Prisma rectangular": PrismaRectangular,
    }

    def __init__(self) -> None:
        super().__init__()
        self.title("Actividad 3 | Figuras geométricas")
        self.geometry("980x720")
        self.minsize(900, 680)
        self.configure(bg=self.BG)
        self._configurar_estilos()
        self.campos: dict[str, ttk.Entry] = {}
```

Código fuente (Python) - Ejercicio 8.3: figuras.py - continuación (3/6)

```
        self._construir_interfaz()
        self._actualizar_campos()
        self._dibujar_figura()

    def _configurar_estilos(self) -> None:
        style = ttk.Style(self)
        style.theme_use("clam")
        style.configure("TFrame", background=self.BG)
        style.configure("TLabel", background=self.BG, foreground=self.NAVY, font=("Segoe UI", 10))
        style.configure("TCombobox", padding=8, font=("Segoe UI", 10))
        style.configure("TEntry", padding=9, fieldbackground="#FBFCFE", font=("Segoe UI", 11))
        style.configure(
            "Primary.TButton", background=self.BLUE, foreground=self.WHITE,
            padding=(15, 10), font=("Segoe UI", 10, "bold"), borderwidth=0
        )
        style.map("Primary.TButton", background=[("active", "#214EAD")])
        style.configure(
            "Secondary.TButton", background="#E9EEF7", foreground=self.NAVY,
            padding=(15, 10), font=("Segoe UI", 10, "bold"), borderwidth=0
        )

    def _construir_interfaz(self) -> None:
        header = tk.Frame(self, bg=self.NAVY, height=112)
        header.pack(fill="x")
        header.pack_propagate(False)
        tk.Label(header, text="Calculadora de figuras", bg=self.NAVY, fg=self.WHITE,
                 font=("Segoe UI", 23, "bold")).pack(anchor="w", padx=30, pady=(20, 2))
        tk.Label(header, text="Selecciona una figura, registra sus dimensiones y calcula sus propiedades.",
                 bg=self.NAVY, fg="#D8E4F7", font=("Segoe UI", 10)).pack(anchor="w", padx=32)

        body = tk.Frame(self, bg=self.BG)
        body.pack(fill="both", expand=True, padx=24, pady=22)
        body.columnconfigure(0, weight=5, uniform="main")
        body.columnconfigure(1, weight=4, uniform="main")
        body.rowconfigure(0, weight=1)

        left = tk.Frame(body, bg=self.WHITE, highlightbackground="#E0E7F0", highlightthickness=1)
        left.grid(row=0, column=0, sticky="nsew", padx=(0, 10))
        right = tk.Frame(body, bg=self.WHITE, highlightbackground="#E0E7F0", highlightthickness=1)
        right.grid(row=0, column=1, sticky="nsew", padx=(10, 0))

        tk.Label(left, text="DATOS DE LA FIGURA", bg=self.WHITE, fg=self.NAVY,
                 font=("Segoe UI", 12, "bold")).pack(anchor="w", padx=22, pady=(22, 5))
        tk.Label(left, text="Figura geométrica", bg=self.WHITE, fg=self.MUTED,
                 font=("Segoe UI", 9)).pack(anchor="w", padx=22, pady=(11, 5))
        self.seleccion = tk.StringVar(value="Cilindro")
        self.combo = ttk.Combobox(left, textvariable=self.seleccion,
                                  values=list(self.DEFINICIONES), state="readonly")
        self.combo.pack(fill="x", padx=22, ipady=2)
        self.combo.bind("<<ComboboxSelected>>", self._cambio_figura)

        self.formulario = tk.Frame(left, bg=self.WHITE)
        self.formulario.pack(fill="x", padx=22, pady=(15, 10))
        self.formulario.columnconfigure(1, weight=1)

        self.resultado_volumen = tk.Label(left, text="Volumen: -- cm³", bg="#F7F9FD", fg=self.BLUE,
                                          font=("Segoe UI", 14, "bold"), anchor="w", padx=14, pady=12)
        self.resultado_volumen.pack(fill="x", padx=22, pady=(12, 6))
        self.resultado_superficie = tk.Label(left, text="Superficie: -- cm²", bg="#F7F9FD", fg=self.BLUE,
                                             font=("Segoe UI", 14, "bold"), anchor="w", padx=14, pady=12)
        self.resultado_superficie.pack(fill="x", padx=22, pady=6)

        botones = tk.Frame(left, bg=self.WHITE)
```

Código fuente (Python) - Ejercicio 8.3: figuras.py - continuación (4/6)

```
        botones.pack(fill="x", padx=22, pady=(18, 8))
        ttk.Button(
            botones, text="Calcular", style="Primary.TButton",
            command=self.calcular
        ).pack(side="left", fill="x", expand=True, padx=(0, 5))
        ttk.Button(
            botones, text="Limpiar", style="Secondary.TButton",
            command=self.limpiar
        ).pack(side="left", fill="x", expand=True, padx=(5, 0))
        tk.Label(left, text="Las dimensiones deben ser positivas y numéricas.", bg=self.WHITE,
                 fg=self.MUTED, font=("Segoe UI", 8)).pack(anchor="w", padx=22, pady=(7, 15))

        tk.Label(right, text="VISTA DE LA FIGURA", bg=self.WHITE, fg=self.NAVY,
                 font=("Segoe UI", 12, "bold")).pack(anchor="w", padx=20, pady=(22, 4))
        tk.Label(
            right, text="Esquema ilustrativo de la figura seleccionada",
            bg=self.WHITE, fg=self.MUTED, font=("Segoe UI", 9),
            wraplength=280, justify="left"
        ).pack(anchor="w", padx=20, pady=(0, 10))
        self.canvas = tk.Canvas(right, width=330, height=270, bg="#F7F9FD", highlightthickness=0)
        self.canvas.pack(fill="both", expand=True, padx=20, pady=(5, 12))
        self.formula_label = tk.Label(
            right, text="V = πr²h     S = 2πr(r + h)", bg=self.WHITE,
            fg=self.MUTED, font=("Consolas", 10), wraplength=300,
            justify="center"
        )
        self.formula_label.pack(fill="x", padx=20, pady=(3, 8))
        self.estado = tk.Label(self, text="Selecciona una figura para comenzar.", bg=self.BG,
                               fg=self.MUTED, font=("Segoe UI", 9), anchor="w")
        self.estado.pack(fill="x", padx=28, pady=(0, 12))

    def _cambio_figura(self, _event: tk.Event | None = None) -> None:
        self._actualizar_campos()
        self._dibujar_figura()
        self._limpiar_resultados()
        self.estado.configure(text="Ingresa las dimensiones de la figura seleccionada.", fg=self.MUTED)

    def _actualizar_campos(self) -> None:
        for widget in self.formulario.winfo_children():
            widget.destroy()
        self.campos.clear()
        for fila, (clave, etiqueta) in enumerate(self.DEFINICIONES[self.seleccion.get()]):
            tk.Label(
                self.formulario, text=etiqueta, bg=self.WHITE, fg=self.NAVY,
                font=("Segoe UI", 9, "bold"), anchor="w"
            ).grid(row=fila, column=0, sticky="w", pady=7, padx=(0, 12))
            entrada = ttk.Entry(self.formulario)
            entrada.grid(row=fila, column=1, sticky="ew", pady=7, ipady=2)
            self.campos[clave] = entrada

    def _limpiar_resultados(self) -> None:
        self.resultado_volumen.configure(text="Volumen: -- cm³")
        self.resultado_superficie.configure(text="Superficie: -- cm²")

    def limpiar(self) -> None:
        for campo in self.campos.values():
            campo.delete(0, tk.END)
        self._limpiar_resultados()
        self.estado.configure(text="Campos limpios. Ingresa las dimensiones.", fg=self.MUTED)
        if self.campos:
            next(iter(self.campos.values())).focus_set()

    def calcular(self) -> None:
```

Código fuente (Python) - Ejercicio 8.3: figuras.py - continuación (5/6)

```
        valores: list[float] = []
        try:
            for clave, entrada in self.campos.items():
                texto = entrada.get().strip()
                etiqueta = dict(self.DEFINICIONES[self.seleccion.get()])[clave]
                if not texto:
                    raise ValueError(f"Debes ingresar: {etiqueta}.")
                try:
                    valor = float(texto.replace(",", "."))
                except ValueError as exc:
                    raise ValueError(f"{etiqueta} debe ser un número válido.") from exc
                if not math.isfinite(valor) or valor <= 0:
                    raise ValueError(f"{etiqueta} debe ser un número mayor que cero.")
                valores.append(valor)
            figura = self.FABRICAS[self.seleccion.get()](*valores)
            volumen = figura.calcular_volumen()
            superficie = figura.calcular_superficie()
        except ValueError as exc:
            messagebox.showerror("Datos inválidos", str(exc), parent=self)
            self.estado.configure(text="Revisa las dimensiones ingresadas.", fg="#B42318")
            return

        self.resultado_volumen.configure(text=f"Volumen: {volumen:,.2f} cm³")
        self.resultado_superficie.configure(text=f"Superficie: {superficie:,.2f} cm²")
        self.estado.configure(text=f"Cálculo completado: {self.seleccion.get()}.", fg="#16803C")

    def _dibujar_figura(self) -> None:
        c = self.canvas
        c.delete("all")
        c.create_text(165, 24, text=self.seleccion.get(), fill=self.NAVY,
                      font=("Segoe UI", 13, "bold"))
        col = "#2F65D5"
        edge = "#214EAD"
        light = "#B9D0FA"
        shape = self.seleccion.get()
        if shape == "Cilindro":
            c.create_oval(95, 78, 235, 118, outline=edge, fill=light, width=2)
            c.create_line(95, 98, 95, 200, fill=edge, width=2)
            c.create_line(235, 98, 235, 200, fill=edge, width=2)
            c.create_arc(95, 180, 235, 220, start=180, extent=180, style="arc", outline=edge, width=2)
            c.create_arc(
                95, 180, 235, 220, start=0, extent=180,
                style="arc", outline=edge, width=2, dash=(4, 3)
            )
            c.create_line(95, 200, 95, 98, fill=edge, width=2)
            c.create_line(235, 200, 235, 98, fill=edge, width=2)
            c.create_text(75, 148, text="h", fill=self.MUTED, font=("Segoe UI", 11, "bold"))
            c.create_text(164, 105, text="r", fill=edge, font=("Segoe UI", 11, "bold"))
        elif shape == "Esfera":
            c.create_oval(82, 65, 248, 230, fill=light, outline=edge, width=2)
            c.create_oval(94, 100, 236, 196, outline=col, width=2, dash=(4, 3))
            c.create_line(165, 148, 226, 112, fill=edge, width=2)
            c.create_text(215, 100, text="r", fill=edge, font=("Segoe UI", 11, "bold"))
        elif shape == "Pirámide cuadrada":
            c.create_polygon(88, 190, 166, 224, 245, 184, 165, 153, fill=light, outline=edge, width=2)
            c.create_polygon(165, 65, 88, 190, 165, 153, fill="#D8E4FA", outline=edge, width=2)
            c.create_polygon(165, 65, 245, 184, 165, 153, fill="#8EAEF0", outline=edge, width=2)
            c.create_line(165, 65, 165, 185, fill=edge, dash=(4, 3), width=2)
            c.create_text(177, 120, text="h", fill=edge, font=("Segoe UI", 10, "bold"))
            c.create_text(205, 191, text="a", fill=edge, font=("Segoe UI", 10, "bold"))
        elif shape == "Cubo":
            c.create_polygon(90, 105, 196, 105, 246, 145, 140, 145, fill="#D8E4FA", outline=edge, width=2)
            c.create_polygon(90, 105, 140, 145, 140, 235, 90, 195, fill="#AFC8F8", outline=edge, width=2)
```

Código fuente (Python) - Ejercicio 8.3: figuras.py - continuación (6/6)

```
            c.create_polygon(140, 145, 246, 145, 246, 235, 140, 235, fill=light, outline=edge, width=2)
            c.create_line(90, 195, 196, 195, fill=edge, width=2)
            c.create_line(196, 105, 196, 195, fill=edge, width=2)
            c.create_line(196, 195, 246, 235, fill=edge, width=2)
            c.create_text(115, 91, text="a", fill=edge, font=("Segoe UI", 11, "bold"))
        else:
            c.create_polygon(65, 100, 205, 100, 260, 138, 120, 138, fill="#D8E4FA", outline=edge, width=2)
            c.create_polygon(65, 100, 120, 138, 120, 220, 65, 182, fill="#AFC8F8", outline=edge, width=2)
            c.create_polygon(120, 138, 260, 138, 260, 220, 120, 220, fill=light, outline=edge, width=2)
            c.create_line(65, 182, 205, 182, fill=edge, width=2)
            c.create_line(205, 100, 205, 182, fill=edge, width=2)
            c.create_text(135, 204, text="largo", fill=edge, font=("Segoe UI", 9, "bold"))

        formulas = {
            "Cilindro": "V = πr²h     S = 2πr(r + h)",
            "Esfera": "V = 4πr³/3     S = 4πr²",
            "Pirámide cuadrada": "V = a²h/3     S = a² + 2al",
            "Cubo": "V = a³     S = 6a²",
            "Prisma rectangular": "V = largo · ancho · altura\nS = 2(la + lh + ah)",
        }
        self.formula_label.configure(text=formulas[shape])


def main() -> None:
    app = VentanaFiguras()
    app.mainloop()


if __name__ == "__main__":
    main()
```

Repositorio en GitHub

https://github.com/JuanMFer24/actividadpoo3